UCD School of Physics



PHYC40600 - Advanced Laboratories

Muon Detector

Dearbhail O'Mahony

October 28, 2026

Abstract

This report aims to experimentally estimate the lifetime of a muon using a muon detector setup. Muons are produced when cosmic rays interact with Earth's atmosphere and are abundant at sea level from this source. A muon detector can be used to track the time between an incident muon being stopped and decaying, and, by detecting a large number of muons, the lifetime can be extracted from the resulting decay curve. Two experimental runs were carried out using alterations of the same detector. Muon lifetime values of 2.098 ± 0.02 μs and 2.036 ± 0.04 μs were found for a 115 hour and 49 hour run respectively. From the longer run, the ratio of positive to negative muons was estimated to be 0.591 ± 0.01.

Contents

1 Introduction **1**
1.1 Cosmic Ray Muons 1
1.2 Positive and Negative Muons 1
1.3 Muon Decay and the Scintillator Detector 1

2 Methodology **2**
2.1 Logic Circuit 2
2.2 Manual Delay 2
2.3 Data Collection and Analysis 2

3 Results **3**
3.1 115-hour Run 3
3.2 49-hour Run 3
3.3 Comparison and Discussion 4

4 Conclusion **4**

References **5**

Appendices **6**

1 Introduction

1.1 Cosmic Ray Muons

Muons are heavy charged particles with a mass of 105.7 Mev/c^2, approximately 207 times that of an electron. They are produced when high-energy particles from astrophysical sources, such as supernovae, collide with the Earth's atmosphere. These incident astrophysical particles are known as primary cosmic rays, while the particle shower resulting from atmospheric interactions is known as a secondary cosmic ray.[1]

Primary cosmic rays consist mainly of atomic nuclei and protons travelling at close to the speed of light. As such, when they collide with atoms in the atmosphere, the energy is sufficient to produce unstable intermediate pions and kaons. The decay of a charged pion then produces a muon, which, at sea level, are the most abundant component of these secondary cosmic rays, with a flux of approximately 70 muons m^{-2} s^{-1} sr^{-1}.[1]

Once produced, cosmic ray muons decay with a literature lifetime of 2.197 μs.[2] This report will focus on an experimental determination of this property, however the the detection of muons has many other applications, including muon radiography, where the measured attenuation of muons can be used to infer the thickness of a material, and muon metrology, which is often used for the calibration and alignment of nuclear and particle physics equipment.[1]

1.2 Positive and Negative Muons

Muons can decay by one of two processes, depending on if they have a positive or negative charge.

$$\mu^- \rightarrow e^- + \bar{\nu}_e + \nu_\mu \quad (1)$$

$$\mu^+ \rightarrow e^+ + \nu_e + \nu_\mu \quad (2)$$

Negative muons can undergo an additional interaction with a proton such that $\mu^- + p \rightarrow n + \nu_\mu$. The quoted muon lifetime value $\tau = 2.197$ μs is therefore slightly lower for μ^- in a material, whereas the μ^+ lifetime, which do not undergo this second interaction mechanism, is the same as in free space. This reduced lifetime for μ^- is dependent on the atomic number of the relevant material, and has been measured to be $\tau^- = 2.043 \pm 0.003$ μs in carbon.[3]

When observing cosmic ray muons, we expect to observe muons of both charges. The observed lifetime should therefore fall somewhere in between $\tau_+ = \tau$ and τ_-. Defining $\rho = N_+/N_-$ to be the ratio of positive to negative muons, we can write an expression for this average lifetime $< \tau >$. [4]

$$< \tau >= (1 + \rho)\frac{\tau_-\tau_+}{\tau_+ + \rho\tau_-} \quad (3)$$

Solving for ρ allows us to estimate the ratio of muon charge from the observed lifetime.

$$\rho = -\frac{\tau_+}{\tau_-}\left(\frac{\tau_- - \tau_{obs}}{\tau_+ - \tau_{obs}}\right) \quad (4)$$

From literature, the expected value of this ratio is approximately 1.2766.[5]

1.3 Muon Decay and the Scintillator Detector

It is possible to measure the time taken for this decay to occur in a large sample of incident muons, and from this to extract an experimental value for the muon lifetime. Using a scintillator detector, a pulse will be sent out when a muons stops in or passes through the detector. If a muon is stopped within the detector, the electron produced from its decay will send out an additional pulse. Hence, measuring the time between two pulses, and counting these over a sufficiently large sample of muons, will give a decay curve of the form Equation (5), where τ is the muon lifetime.

$$I(t) = I_0 e^{-t/\tau} \quad (5)$$

As the decay electrons can only be detected if this decay occurs within the scintillator, only those muons that are stopped by the scintillator are of interest in this experiment, not those that pass through. The methods used to ensure that only the former are detected are discussed in Section 2. Physically, whether a muon is stopped by a scintillator detector is determined by the range of the muon and the linear stopping power of the scintillator material. The determining factor for this, when comparing only muons in a known scintillator, is then the initial kinetic energy of the muon, before entering the scintillator. Muons below a critical energy will be stopped by the detector and can therefore have their decay measured.[1]

2 Methodology

A scintillator detector was connected to a photomultiplier tube (PMT) which was in turn connected to a high voltage source. From the PMT, pulses were sent to a Phillips Quad Bipolar DC 300 Mhz Amplifier, and from there to a CAEN DT1081B Desktop Four Fold Programmable Logic Unit. From this unit, start and stop pulses were sent to an ORTEC MODEL 566 Time-to-Amplitude Converter (TAC). A pulse with an amplitude corresponding to the length of the start-stop time, as determined by the TAC, was then sent to a potentiometer, where the output was divided by a factor of 4, before being sent to a multi-channel analyser. From there the data was recoded by a PC as counts per channel, with each channel corresponding to an interval of time.

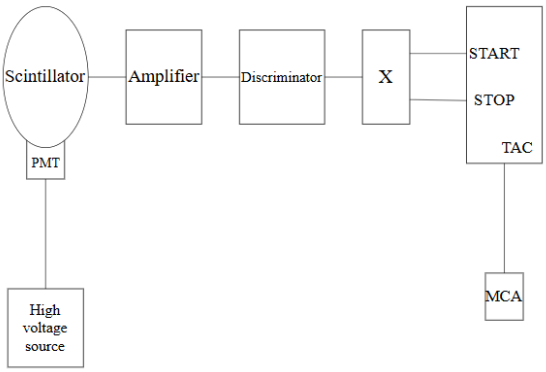


Figure 1: A diagram of the muon detector setup. Muons and their decay electrons trigger pulses from a scintillator, which travel from there to an amplifier, a discriminator, and a TAC. The component labelled *'X'* is controll signals to the TAC. Two methods were used for this component, detailed in sections 2.1 and 2.2.

2.1 Logic Circuit

Using three sections of the *DT1081B*, a logic circuit was constructed to increase the accuracy of the TAC in detecting only those time intervals that originated from muons entering the scintillator and decaying within to release an electron. As this logic took the place of the component marked X in Figure (1), input to section A of the *DT1081B* came from the amplifier. This section was set to the *'fan out'* function with a discriminator input at a setting of -200 mV. This produced equal outputs of 15 ns pulses going to sections B and C when the discriminator was triggered.

At the input of section B, also in the *'fan out'* function, now with a standard NIM 300 mV input, a gate of 10 μs. No delay setting was found to be necessary, as the delay introduced by the circuit was sufficient. Additional 15 ns output pulses from here went to the TAC *START* input and to a second section C input.

Finally, section C was set to the *'and'* function, with a 15 ns gate at a 10 ns delay at the input from A. A 15 ns output pulse from section C went to the TAC *STOP* input. Thus, there must be a valid start and stop pulse within the expected maximum time interval for the TAC to register a start-stop time. This minimises the counting of muons that pass through but are not stopped by the scintillator.

2.2 Manual Delay

An alternative, simpler system was constructed using only sections A and B of the *DT1081B*. The input to section A was as before, and this section remained in the *'fan out'* function, with the discriminator input now set to -150 mV. One output was sent directly to the TAC *STOP*, while the other was sent to section B. This input was set to a delay of 5 ns. The setup now closely follows that of [6], with a programmed time delay replacing the physical delay using a longer cable.

For this experimental setup, both output pulses to the TAC were set to a width of 100 ns. This was done in an effort to avoid double-counting of pulses due to ringing in the pulse signal, which was evident in the data collected from the conditions in Section 2.1 as a large peak in the very early channels dominating the decay curve.

2.3 Data Collection and Analysis

Two rounds of data collection were completed: a 115-hour run with the setup described in Section 2.1, and a 49-hour run as described in Section 2.3. For both runs, the high voltage source was set to 1700 V.

A calibration was carried out to determine the time width of the MCA channels. Section B of the *DT1081B* was set to the *'pulse generator'* function with a pulse width of 20 nm. These pulses were output to the amplifier and from here the setup was as described in 2.1 and Figure (1). Pulse periods between 0.5 and 9 μm were used, and at each period 30 seconds of data was collected. A plot of

time period in μs against the peak channel number as constructed, with the uncertainty on the points taken to be the width of the peak in channels. A linear fit was made to this plot and the fitting parameters were used to convert channel number to time for the subsequent analysis.

As the data collected is in the form of the discrete counts, it follows Poisson statistics. With the MCA calibration providing a channel width of 2 ± 0.7 ns and the data being of the form of an exponential decay, many channels towards the higher time range had low or no counts. In order to fit a curve to this data, it must first be binned so that there is a sufficient number of expected counts E_i per bin. It was chosen to have $E_i > 10$ per bin due to the large number of bins in the initial data, as a larger E_i better avoids skewing the data.[7] Variable bin widths were selected to fulfil this criteria, with the expected counts given by the function $60 * exp(-t/2.197) + 2$, where 2.197 is the literature value for τ, and the other parameters were chosen by eye to fit the data.

As mentioned previously, the 115-hour run data was dominated by a large, sharp peak in the 100 ns range. This was assumed to be primarily due to the output pulse not being wide enough to adequately account for double counting from ringing in the pulse signal. As such, this peak was disregarded so that the muon decay curve could be properly analysed. The starting channel range affected by this peak was identified, and an initial decay fit was made to the remaining data. This fit was applied to the full data set and channels were excluded if the data point fell outside of the average distance from this fit.

Although the adjustment to the output pulse width was effective in reducing the size of this peak for the 49-hour run data, there was still a reduced peak present such that it was necessary to repeat this procedure for the second set of data.

For the final decay curve fit, an additional fitting parameter B was added to the equation. This was to account for an constant background level in the data. We assume this background to be primarily due to other sources in the lab as well as multiple muons passing through the scintillator within the required time frame. As these events are expected to be random and uncorrelated, the resulting background can be assumed to be constant over time for a long observation period relative to this time frame.

$$I(t) = I_0 e^{-t/\tau} + B \tag{6}$$

This fit was applied to both rounds of data using the Scipy *'curve_fit'* function, with the error on counts given by $\sqrt{E_i}$. Errors on the lifetime were then taken from the resulting covariant matrix.

3 Results

3.1 115-hour Run

Figure (2) shows the resulting muon decay curve from the 115-hour data collection run. Analysis of this data provides a muon lifetime of $\tau = 2.098 \pm 0.02$ μs. This is not within experimental uncertainty of the literature value, however as discussed in Section 1.2, the observed value is expected to be somewhat lowered due to the additional interaction mechanism of μ^-.

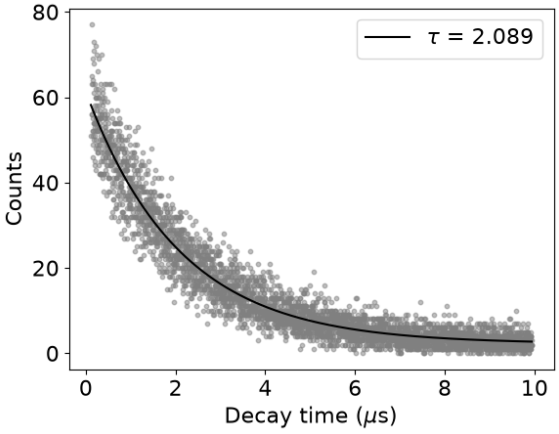


Figure 2

Calculating the ratio of positive to negative muons with this value, using equation (4), gives a value of $\rho = 0.591 \pm 0.01$. This does not agree with the literature, which indicates that we expect a value of $\rho > 1$, in other words more positive than negative muons. [5] This suggests that the observed lifetime value is lower than what can be explained by the effects of negative muons, and may have been affected by other experimental errors, such as background noise in the lab. From the fit, a background level of 2.190 ± 0.11 counts per channel is found.

3.2 49-hour Run

The results of the 49-hour run after analysis are shown in Figure (3). For this data set, a lifetime

of $\tau = 2.036 \pm 0.04$ μs is obtained. This is significantly lower than the literature value. As this value no longer falls between the literature τ_+ and τ_-, it no longer makes sense to estimate a value of ρ using equation (4).

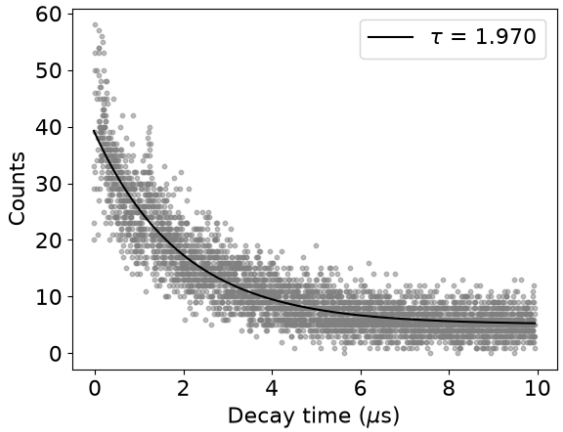


Figure 3

For this run, there is a higher fitted background of 4.944 ± 0.10 counts per channel. To demonstrate the necessity of carrying out the early peak elimination in this data set in addition to the longer run, a fit was made to the binned but unrefined data set was made and found the lower lifetime of 1.882 ± 0.04 μs. Although this is a less pronounced effect than was viewed in the longer run before peak detection, where the decay curve was not visible in the raw data, there is still a significant effect on the lifetime value.

3.3 Comparison and Discussion

From this analysis, neither data run was able to accurately recreate the literature muon lifetime. However, comparing the lifetimes obtained from the two data runs, the first gives a significantly more accurate result. Given that there were multiple alterations in the setup, with both the system for input to the TAC, and the discriminator setting, being altered, an exact reasoning for this cannot be given. However, it is likely that the single biggest factor is the data collection time, as the first run was more than double the length. It is also possible that the combination of a lowering the discriminator setting and removing the more rigid logic circuit resulted in a larger volume of false or background counts caused by multiple muons passing through the scintillator. This is indicated by the higher background in the second run, which, as calculated by fitting Equation (5), is more than doubled from the first run.

Although the primary aim of the alterations for the second run was to reduce the dominating peak effect in the early channels, a sufficient peak was still present in the data to effect the resulting lifetime by over 0.15 μs, or 7% of the literature value. Thus, analysis was still required to eliminate this peak, and it can be concluded that the first setup was more effective, with the potential for improvement by increasing the output pulse width. Overall, the need for more rigorous testing of the effect of individual factors in this experiment is indicated and would be an interesting topic of future study.

4 Conclusion

A muon detector was successfully used to detect cosmic ray muons and their decay electrons. Two runs of data collection were completed to detect a large sample of muon decay times in order to find the muon lifetime, which is expected to be $\tau = 2.197$ μs.[2] A 115-hour run used a higher discriminator setting, a logic circuit, and narrower output pulses to the TAC. A 49-hour run used a lower discriminator setting, a simple time delay, and wider output pulses. After data analysis, including binning according to Poisson statistics and fitting a constant background, lifetime values were found to be 2.098 ± 0.02 μs and 2.036 ± 0.04 μs. For the longer run a corresponding μ_+/μ_- ratio of 0.591 ± 0.01 was calculated. Neither run matched the literature literature value, however the 115-hour run came closer, likely due to the longer detection time, but possibly in part due to the use of a logic circuit and a higher discriminator setting. In a future application of this setup it would be useful to more thoroughly examine the individual effects of these settings.

References

[1] G. Bonomi et al. "Applications of cosmic-ray muons". In: *Progress in Particle and Nuclear Physics* 112 (May 2020), p. 103768. DOI: 10.1016/j.ppnp.2020.103768. URL: https://www.sciencedirect.com/topics/physics-and-astronomy/primary-cosmic-ray.

[2] T.P. Gorringe and D.W. Hertzog. "Precision muon physics". In: *Progress in Particle and Nuclear Physics* 84 (2015), pp. 73–123. DOI: 10.1016/j.ppnp.2015.06.001.

[3] R. A. Reiter et al. "Precise Measurements of the Mean Lives of+andMesons in Carbon". In: *Physical Review Letters* 5.1 (1960), pp. 22–23. DOI: 10.1103/physrevlett.5.22.

[4] T Coan and J Ye. *Muon Physics*. URL: https://www.physics.rutgers.edu/~eandrei/389/muon/muon-new-F15.pdf.

[5] A. Calderón for the CMS Collaboration. "Measurement of the charge ratio of atmospheric muons with the CMS detector". In: *Astrophysics and Space Sciences Transactions* 7.1 (Mar. 2011), pp. 79–83. DOI: 10.5194/astra-7-79-2011. URL: https://arxiv.org/pdf/1005.5332.pdf.

[6] R. E. Hall, D. A. Lind, and R. A. Ristinen. "A Simplified Muon Lifetime Experiment for the Instructional Laboratory". In: *American Journal of Physics* 38.10 (Oct. 1970), pp. 1196–1200. DOI: 10.1119/1.1976002.

[7] Ifan Hughes and Thomas P A Hase. *Measurements and their uncertainties : a practical guide to modern error analysis*. Oxford Oxford University Press, 2011. ISBN: 9780199566334.

Appendices

Appendix 1

Full Jupyter notebook used for analysis.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from scipy.optimize import curve_fit
from scipy.stats import poisson
```

<h4 id="Calibration:">Calibration:</h4>

In [52]:
```
#loading files for calibration:
cal005 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/calibration/cal_0-5mus.txt')
cal02 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/calibration/cal_2mus.txt')
cal03 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/calibration/cal_3mus.txt')
cal04 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/calibration/cal_4mus.txt')
cal05 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/calibration/cal_5mus.txt')
cal06 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/calibration/cal_6mus.txt')
cal07 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/calibration/cal_7mus.txt')
cal08 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/calibration/cal_8mus.txt')
cal09 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/calibration/cal_9mus.txt')

calibration_dfs = [cal005, cal02, cal03, cal04, cal05, cal06, cal07, cal08, cal09]
```

In [45]:
```
#defining a function to find the channel in which the peak occurs, and the width on either side for errors
def get_channel_err(df):
    peak = np.max(df['0'])
    channel_no = np.where(df['0']==peak)[0][0]

    peak_area = df[df['0']>0]

    low_err = channel_no - peak_area.index[0]
    high_err = peak_area.index[-1] - channel_no

    return channel_no, low_err, high_err
```

In [46]:
```
channels = []
cal_low_errs = []
cal_high_errs = []

for df in calibration_dfs:
    channel_no, low_err, high_err = get_channel_err(df)
    channels.append(channel_no)
    cal_low_errs.append(low_err)
    cal_high_errs.append(high_err)

times = [0.5, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [47]:
```
#taking mean of asymmetric peak width error, as symmetric errror needed for fitting
means = []
for x, y in zip(cal_low_errs, cal_high_errs):
    means.append(x+y/2)
```

In [48]:
```
#fitting a calibration line to the data
def lin(x, a, b):
    return a*x + b

popt, pcov = curve_fit(lin, channels, times, sigma=means)
a_time, b_time = popt
a_err, b_err = np.sqrt(np.diag(pcov))
print(a_time, b_time)
```

```
0.0025127520906402002 -0.03913971725302579
```

In [49]:
```
#plotting calibration line
fig, ax = plt.subplots()

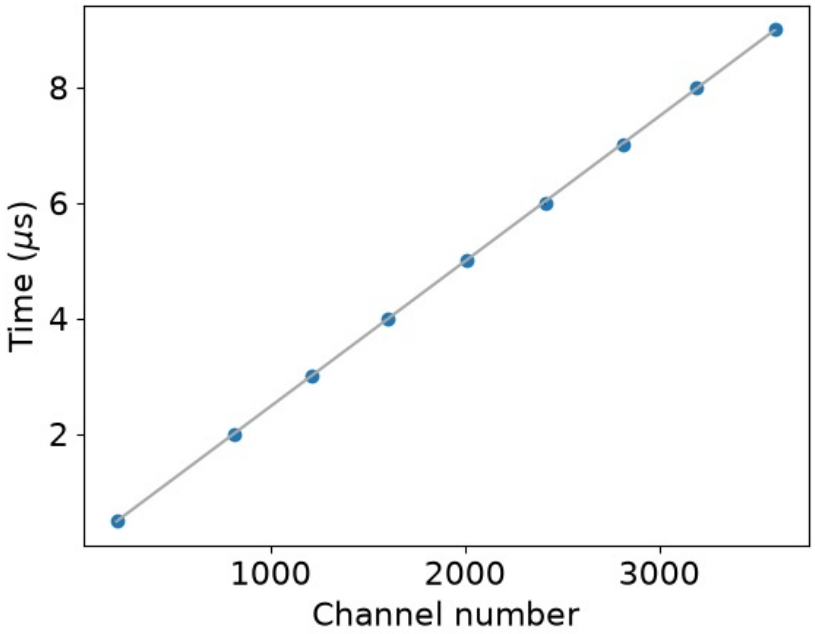
ax.scatter(channels, times)

cal_err = [cal_low_errs, cal_high_errs]
ax.errorbar(channels, times, xerr = cal_err, fmt='None', color='black')

plt.plot(channels, lin(np.array(channels), a_time, b_time), color='darkgray')
```

```
ax.set_xlabel('Channel number')
ax.set_ylabel(r'Time ($\mu$s)')
```

Out[49]: Text(0, 0.5, 'Time ($\\mu$s)')



In [50]:
```
def channel_time(channel_no):
    return a_time*channel_no + b_time

def time_channel(time):
    return (time - b_time)/a_time

#finding temporal width of channels
twidth = channel_time(4096)/4096
twidth_err = np.sqrt(a_err**2+b_err**2)

print(twidth, twidth_err)
```

```
0.0025031964956077233 0.007596350892967363
```

<h4 id="Run-1-analysis:">Run 1 analysis:</h4>

In [8]:
```
#defining a decay function to fit to the data, with b = lifetime, c = constant background term
def decay(x, a, b, c):
    return a*np.exp(-x/b) + c
```

In [9]:
```
data1 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/thurs_15_0115.txt', header=None)
```

In [10]:
```
data1.index = channel_time(data1.index)
```

In [11]:
```
#rebinning data so that expected counts >10 per bin
channels = np.array(data1.index)
counts = data1[0].values

bin_centres = []
bin_averages = []
expected_averages = []

#expected counts calculated from literature lifetime and other parameters fit by eye to decay
exp=decay(channels, 60, 2.197, 2)

i = len(channels)-1
while i>0:
    exp_totals = exp[i]
    count_totals = counts[i]
    n=0
    #combining channels until expected is >10
    while exp_totals<10:
        n+=1
        exp_totals+=exp[i-n]
        count_totals+=counts[i-n]

    #taking average and centre value if bin encompasses more than one channel,
    #otherwise taking channel and count as normal
    if n>0:
        bin_centres.append((channels[i] + channels[i-n])/2)
```

```
        bin_averages.append(count_totals/(n+1))
        expected_averages.append(exp_totals/(n+1))
    else:
        bin_centres.append(channels[i])
        bin_averages.append(count_totals)
        expected_averages.append(exp_totals)
    i-=n+1
```

In [12]:
```
data1_binned = pd.DataFrame({'bin_centre':bin_centres, 'bin_avg':bin_averages,
                             'expected_avg':expected_averages})
data1_binned.set_index('bin_centre', inplace=True)
```

In [13]:
```
#cutting first few channels by eye to get a desire fit
data_fit = data1_binned[data1_binned.index>channel_time(70)]
```

In [54]:
```
#fitting preliminary curve so that peak can be removed
popt, pcov = curve_fit(decay, data_fit.index, data_fit['bin_avg'], p0 = [1, 1, 1],
                       sigma=np.sqrt(data_fit['expected_avg'].values))
a_prelimfit, b_prelimfit, c_prelimfit = popt
a_prelimfit_err, b_prelimfit_err, c_prelimfit_err = np.sqrt(np.diag(pcov))
```

In [55]:
```
print(a_prelimfit, b_prelimfit, c_prelimfit)
```

```
58.71373982888074 2.1535569569425888 1.843537932157304
```

In [56]:
```
print(a_prelimfit_err, b_prelimfit_err, c_prelimfit_err)
```

```
0.4404509459146034 0.02446632095236657 0.10751789747136009
```

In [57]:
```
exclude_channels = []
for index, point in zip(data1_binned.index, data1_binned['bin_avg']):
    dist = np.abs(point - decay(index, a_prelimfit, b_prelimfit, c_prelimfit))
    if dist>20:
        exclude_channels.append(index)
    if index>9.94:
        exclude_channels.append(index)
```

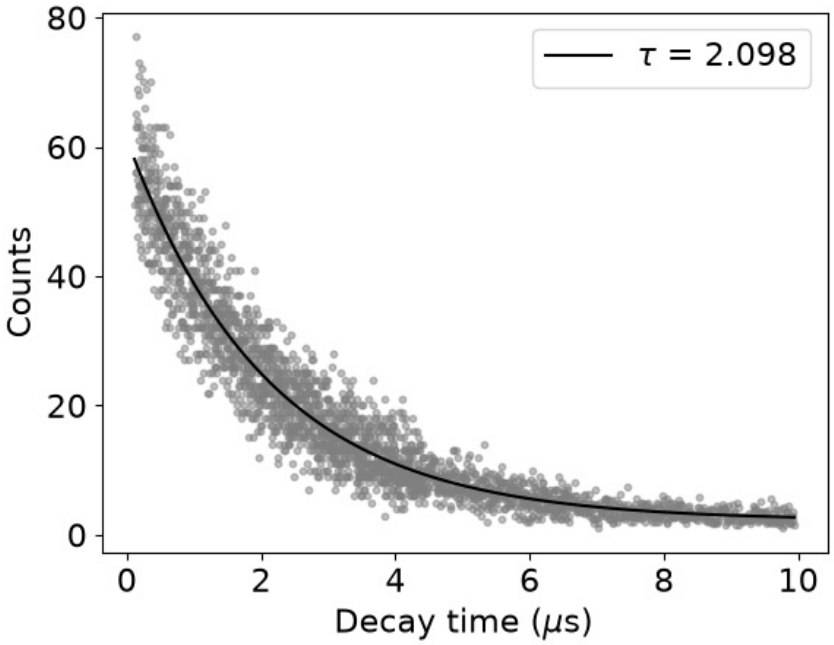
In [58]:
```
data1_ref = data1_binned.drop(exclude_channels)
```

In [59]:
```
popt, pcov = curve_fit(decay, np.array(data1_ref.index), data1_ref['bin_avg'].values, p0 = [1, 1, -1],
                       sigma = np.sqrt(data1_ref['expected_avg'].values))
a1, b1, c1 = popt
a1_err, b1_err, c1_err = np.sqrt(np.diag(pcov))
print(a1, b1, c1)
print(a1_err, b1_err, c1_err)
```

```
59.01357247444953 2.097627368714904 2.1900034794459993
0.4375516079477065 0.023734754587873122 0.10837825070261944
```

In [20]:
```
plt.rcParams.update({'font.size': 16})
```

In [21]:
```
plt.scatter(data1_ref.index, data1_ref['bin_avg'], s=10, color='gray', alpha=0.5)
plt.plot(data1_ref.index, decay(data1_ref.index, a1, b1, c1), color='black',
         label=r'$\tau$ = '+str(np.round(b1, decimals=3)))
plt.xlabel(r'Decay time ($\mu$s)')
plt.ylabel('Counts')
plt.legend()
plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/figures/datarun1_decaycurve.png",
            bbox_inches='tight')
```



In [22]:
```
tpos = 2.197
tneg = 2.043
```

In [23]:
```
def muonratio(tobs):
    return -(tpos/tneg)*((tneg-tobs)/(tpos-tobs))

muonratio(b1)
```

Out[23]: np.float64(0.5911601934953676)

<h4 id="Run-2-analysis:">Run 2 analysis:</h4>

In [25]:
```
#repeating process for second data run
data2 = pd.read_csv(
    'C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/data/tues_20_0049.txt', header=None)
```

In [26]:
```
data2.index = channel_time(data2.index)
```

In [27]:
```
channels = np.array(data2.index)
counts = data2[0].values

bin_lower_edges = []
bin_upper_edges = []
channel_sizes = []
bin_centres = []
bin_averages = []

exp=decay(channels, 60, 2.197, 2)

i = len(channels)-1
while i>0:
    exp_totals = exp[i]
    count_totals = counts[i]
    n=0
    while exp_totals<10:
        n+=1
        exp_totals+=exp[i-n]
        count_totals+=counts[i-n]

    bin_lower_edges.append(channels[i])
    bin_upper_edges.append(channels[i-n])
    channel_sizes.append(n+1)

    if n>0:
        bin_centres.append((channels[i] + channels[i-n])/2)
        bin_averages.append(count_totals/(n+1))
    else:
        bin_centres.append(channels[i])
        bin_averages.append(count_totals)
    i-=n+1
```

In [28]:
```
data2_binned = pd.DataFrame({'bin_centre':bin_centres, 'bin_avg':bin_averages,
                             'expected_avg':expected_averages})
data2_binned.set_index('bin_centre', inplace=True)
```

In [29]:
```
data2_fit = data2_binned[data2_binned.index>channel_time(40)]
```

In [41]:
```
popt, pcov = curve_fit(decay, data2_fit.index, data2_fit['bin_avg'], p0 = [1, 1, 1])
a_prelimfit, b_prelimfit, c_prelimfit = popt
a_prelimfit_err, b_prelimfit_err, c_prelimfit_err = np.sqrt(np.diag(pcov))
```

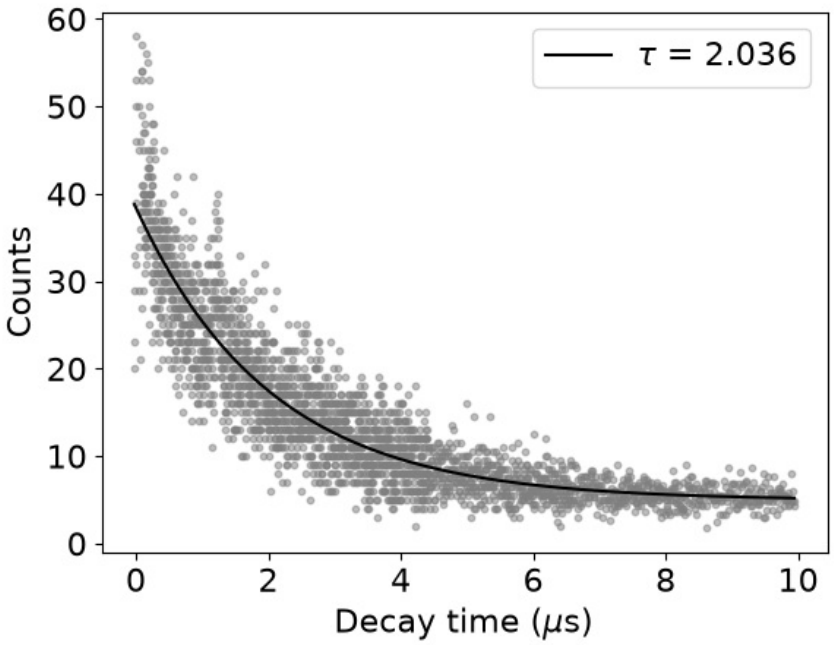
In [31]:
```
exclude_channels = []
for index, point in zip(data2_binned.index, data2_binned['bin_avg']):
    dist = np.abs(point - decay(index, a_prelimfit, b_prelimfit, c_prelimfit))
    if dist>20:
        exclude_channels.append(index)
    if index>9.94:
        exclude_channels.append(index)
```

In [32]:
```
data2_ref = data2_binned.drop(exclude_channels)
```

In [34]:
```
popt, pcov = curve_fit(decay, data2_ref.index, data2_ref['bin_avg'].values, p0 = [80, 2, 2], sigma=np.sqrt(data
a2, b2, c2 = popt
a2_err, b2_err, c2_err = np.sqrt(np.diag(pcov))
print(a2, b2, c2)
print(a2_err, b2_err, c2_err)
```

```
33.473477914558515 2.0356172199009457 4.943577737598522
0.3905804961105231 0.03660866147935818 0.09787338173743897
```

In [35]:
```
plt.scatter(data2_ref.index, data2_ref['bin_avg'].values, s=10, color='gray', alpha=0.5)
plt.plot(data2_ref.index, decay(data2_ref.index, a2, b2, c2), color='black', label=r'$\tau$ = '+str(np.round(b2
plt.xlabel(r'Decay time ($\mu$s)')
plt.ylabel('Counts')
plt.legend()
plt.savefig("C:/Users/dearb/Documents/UCD_labs/fourth_year_labs/muon_detector/figures/datarun2_decaycurve.png",
```



In [ ]: